Tutorial: Asynchrone Programmierung mit async und await

Dieses Tutorial baut auf ein Tutorial von Microsoft auf, das unter folgender Adresse zu finden war:

https://learn.microsoft.com/de-de/dotnet/csharp/asynchronous-programming/

Vorbereitung

Legen Sie ein neues Projekt für eine Konsolenanwendung unter .NET an. Legen Sie unterhalb der Klasse `Program` die folgenden leeren Klassen an:

```
class Egg { }

class Juice { }

class Toast { }

class Bacon { }

class Coffee { }
```

Diese Klassen dienen später nur als Rückgabetyp für die Methoden.

Einleitung

Das aufgabenbasierte asynchrone Programmiermodell stellt eine Abstraktion über asynchronen Code bereit. Sie können Code in gewohnter Weise als eine Folge von Anweisungen schreiben. Sie können diesen Code so lesen, als ob jede Anweisung abgeschlossen wäre, bevor die nächste Anweisung beginnt. Der Compiler führt eine Reihe von Transformationen durch, da möglicherweise einige dieser Anweisungen gestartet werden und eine Task zurückgeben, die die derzeit ausgeführte Arbeit darstellt.

Dies ist das Ziel dieser Syntax: Code zu aktivieren, der sich wie eine Folge von Anweisungen liest, aber in einer deutlich komplizierteren Reihenfolge ausgeführt wird, die auf einer externen Ressourcenzuordnung und dem Abschluss von Aufgaben basiert. Vergleichbar ist dies mit der Art und Weise, wie Menschen Anweisungen für Prozesse erteilen, die asynchrone Aufgaben enthalten. In diesem Artikel verwenden Sie als Beispiel Anweisungen für die Zubereitung eines Frühstücks, um zu lernen, wie die Schlüsselwörter async und await die Analyse von Code erleichtern, der eine Reihe asynchroner Anweisungen enthält. Um die Zubereitung eines Frühstücks zu erläutern, würden Sie Anweisungen schreiben, und Ihre Liste sähe ungefähr so aus:

1. Schenken Sie sich eine Tasse Kaffee ein.
2. Erhitzen Sie eine Pfanne, und braten Sie darin zwei Eier.
3. Braten Sie drei Scheiben Frühstücksspeck.
4. Toasten Sie zwei Scheiben Brot.
5. Bestreichen Sie das getoastete Brot mit Butter und Marmelade.
6. Schenken Sie sich ein Glas Orangensaft ein.

Wenn Sie über Erfahrung im Kochen verfügen, würden Sie diese Anweisungen asynchron ausführen. Sie würden zunächst die Pfanne für die Eier erhitzen und dann mit dem Frühstücksspeck beginnen. Sie würden das Brot in den Toaster stecken und danach mit den Eiern beginnen. Bei jedem Schritt des Prozesses würden Sie eine Aufgabe starten und dann Ihre Aufmerksamkeit auf Aufgaben lenken, die für Ihre Aufmerksamkeit bereit sind.

Die Zubereitung eines Frühstücks ist ein gutes Beispiel für asynchrone Arbeiten, die nicht parallel ausgeführt werden. Eine Person (oder ein Thread) kann alle diese Aufgaben erledigen. Um beim Beispiel des Frühstücks zu bleiben: Eine Person kann das Frühstück asynchron zubereiten, indem sie die nächste Aufgabe startet, bevor die erste Aufgabe abgeschlossen ist. Die Zubereitung schreitet voran, und zwar unabhängig davon, ob jemand ein Auge darauf hat oder nicht. Sobald Sie damit beginnen, die Pfanne für die Eier zu erhitzen, können Sie mit dem Braten des Frühstücksspecks beginnen. Nachdem Sie das Braten des Frühstücksspecks begonnen haben, können Sie das Brot in den Toaster stecken.

Für einen parallelen Algorithmus bräuchten Sie mehrere Köche (bzw. Threads). Ein Koch würde sich um die Eier kümmern, ein weiterer um den Frühstücksspeck usw. Jeder Koch würde sich nur auf diese eine Aufgabe konzentrieren. Jeder Koch (bzw. Thread) würde synchron blockiert, während er darauf wartet, dass der Frühstücksspeck gewendet werden muss oder der Toaster das Brot auswirft.

Synchroner Ansatz

Legen Sie nun zunächst einmal die Methoden, welche die einzelnen Arbeitsschritte simulieren sollen, in der Klasse `Program` an:

```
private static Juice PourOJ() {
    Console.WriteLine("Pouring orange juice");
    return new Juice();
}

private static void ApplyJam(Toast toast) {
    Console.WriteLine("Putting jam on the toast");
}

private static void ApplyButter(Toast toast) {
    Console.WriteLine("Putting butter on the toast");
}

private static Toast ToastBread(int slices) {
    for (int slice = 0; slice < slices; slice++) {
        Console.WriteLine("Putting a slice of bread in the toaster");
    }
    Console.WriteLine("Start toasting...");
    Task.Delay(3000).Wait();
    Console.WriteLine("Remove toast from toaster");
    return new Toast();
}

private static Bacon FryBacon(int slices) {
    Console.WriteLine($"putting {slices} slices of bacon in the pan");
    Console.WriteLine("cooking first side of bacon...");
    Task.Delay(3000).Wait();
    for (int slice = 0; slice < slices; slice++) {
        Console.WriteLine("flipping a slice of bacon");
    }
    Console.WriteLine("cooking the second side of bacon...");
    Task.Delay(3000).Wait();
    Console.WriteLine("Put bacon on plate");
    return new Bacon();
}

private static Egg FryEggs(int howMany) {
    Console.WriteLine("Warming the egg pan...");
    Task.Delay(3000).Wait();
    Console.WriteLine($"cracking {howMany} eggs");
    Console.WriteLine("cooking the eggs ...");
    Task.Delay(3000).Wait();
    Console.WriteLine("Put eggs on plate");
    return new Egg();
}
```

```
private static Coffee PourCoffee() {
    Console.WriteLine("Pouring coffee");
    return new Coffee();
}
```

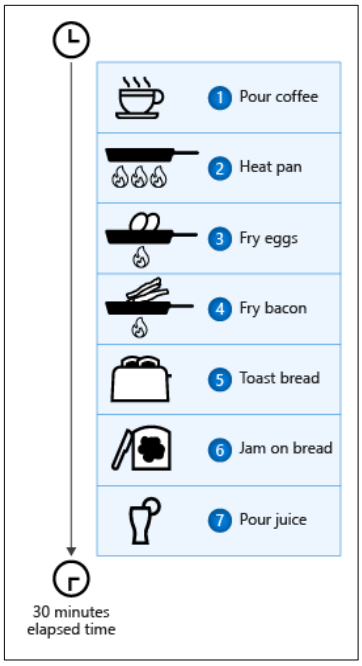
Als nächstes ergänzen Sie Ihre Main-Methode um die folgenden Methodenaufrufe:

```
static void Main(string[] args) {

    Stopwatch sw = new Stopwatch();
    sw.Start();

    Coffee cup = PourCoffee();
    Console.WriteLine("coffee is ready");

    Egg eggs = FryEggs(2);
    Console.WriteLine("eggs are ready");

    Bacon bacon = FryBacon(3);
    Console.WriteLine("bacon is ready");

    Toast toast = ToastBread(2);
    ApplyButter(toast);
    ApplyJam(toast);
    Console.WriteLine("toast is ready");

    Juice oj = PourOJ();
    Console.WriteLine("oj is ready");

    sw.Stop();

    Console.WriteLine($"Breakfast is ready: {sw.ElapsedMilliseconds} ms");
}
```

Sie werden für den Code die folgenden using-Anweisungen benötigen:

```
using System;
using System.Diagnostics;
using System.Threading.Tasks;
```

Starten Sie nun Ihren Code. Alle Aufgaben werden synchron nacheinander abgearbeitet. In der Realität hätte die Zubereitung des Frühstücks ungefähr 30 Minuten gedauert.



Computer interpretieren diese Anweisungen anders als Menschen. Nach jeder Anweisung blockiert der Computer das weitere Vorgehen, bis die Arbeit abgeschlossen ist. Erst danach fährt er mit der

nächsten Anweisung fort. So käme kein schmackhaftes Frühstück zustande. Die späteren Aufgaben würden erst gestartet, wenn die früheren Aufgaben abgeschlossen sind. Die Zubereitung des Frühstücks würde wesentlich länger dauern, und einige Komponenten wären bereits wieder kalt, bis sie serviert werden.

Wenn der Computer die obigen Anweisungen asynchron ausführen soll, müssen Sie asynchronen Code schreiben.

Diese Überlegungen sind wichtig für die Programme, die Sie heutzutage schreiben. Wenn Sie Client-Programme schreiben, möchten Sie, dass die Benutzeroberfläche auf Benutzereingaben reagiert. Ihre Anwendung sollte nicht den Eindruck erwecken, dass sich das Smartphone aufgehängt hat, während es Daten aus dem Web herunterlädt. Wenn Sie Serverprogramme schreiben, möchten Sie nicht, dass Threads blockiert werden. Diese Threads könnten für andere Anforderungen benötigt werden. Die Verwendung von synchronem Code, wenn asynchrone Alternativen vorhanden sind, beeinträchtigt Ihre Möglichkeiten für günstigere Erweiterungen. Sie bezahlen für die blockierten Threads.

Erfolgreiche moderne Anwendungen erfordern asynchronen Code. Ohne Sprachunterstützung erforderte das Schreiben von asynchronem Code Rückrufe, Abschlussereignisse oder andere Methoden, die die ursprüngliche Absicht des Codes verdeckten. Der Vorteil des synchronen Codes besteht darin, dass seine schrittweisen Aktionen das Scannen und Verstehen erleichtern. Bei traditionellen asynchronen Modellen mussten Sie sich auf die asynchronen Eigenschaften des Codes und nicht auf die grundlegenden Aktionen des Codes konzentrieren.

Nicht blockieren, stattdessen „await“ verwenden

Der obige Code zeigt eine schlechte Praxis: das Erstellen von synchronem Code zum Ausführen asynchroner Vorgänge. In der vorliegenden Form hindert dieser Code den Thread an der Ausführung aller anderen Arbeiten. Es wird nicht unterbrochen, während eine der anderen Aufgaben ausgeführt wird. Dies wäre so, als würden Sie den Toaster anstarren, nachdem Sie das Brot hineingesteckt haben. Sie wären so lange für niemanden ansprechbar, bis das getoastete Brot ausgeworfen wurde.

Aktualisieren wir also diesen Code so, dass der Thread nicht blockiert wird, während Aufgaben ausgeführt werden. Das Schlüsselwort await bietet die Möglichkeit, eine Aufgabe zu starten und dann die Ausführung fortzusetzen, wenn diese Aufgabe abgeschlossen ist, ohne dass es dabei zu einer Blockierung kommt. Eine einfache asynchrone Version des Codes für die Frühstückszubereitung sähe daher wie der folgende Codeausschnitt aus:

```
static async Task Main(string[] args) {

    Stopwatch sw = new Stopwatch();
    sw.Start();

    Coffee cup = PourCoffee();
    Console.WriteLine("coffee is ready");

    Egg eggs = await FryEggsAsync(2);
    Console.WriteLine("eggs are ready");

    Bacon bacon = await FryBaconAsync(3);
    Console.WriteLine("bacon is ready");

    Toast toast = await ToastBreadAsync(2);
    ApplyButter(toast);
    ApplyJam(toast);
    Console.WriteLine("toast is ready");
```

```
    Juice oj = PourOJ();
    Console.WriteLine("oj is ready");

    sw.Stop();

    Console.WriteLine($"Breakfast is ready: {sw.ElapsedMilliseconds} ms");
}
```

Damit das Programm lauffähig bleibt, müssen Sie auch die drei Methoden FryEggs, FryBacon und ToastBread ein wenig überarbeiten und umbenennen. Beispielhaft wird dies hier an der Methode FryEggs demonstriert:

```
private static async Task<Egg> FryEggsAsync(int howMany) {
    Console.WriteLine("Warming the egg pan...");
    await Task.Delay(3000);
    Console.WriteLine($"cracking {howMany} eggs");
    Console.WriteLine("cooking the eggs ...");
    await Task.Delay(3000);
    Console.WriteLine("Put eggs on plate");
    return new Egg();
}
```

Die Methode wird von `FryEggs` in `FryEggsAsync` umbenannt. Gleichzeitig wird der Rückgabetyp von `Egg` zu `Task<Egg>` geändert und um das Schlüsselwort `async` ergänzt. Schließlich werden die beiden Zeilen mit dem Aufruf `Task.Delay()` um das Schlüsselwort `await` erweitert. Dafür entfällt dann hier jeweils der Methodenaufruf `Wait()`.

Verfahren Sie mit den Methoden `FryBacon` und `ToastBread` analog. Starten Sie das Programm nun erneut. Sie werden so noch keine Verbesserung bei der Gesamtzeit beobachten.

Dieser Code verursacht keine Blockierung, während die Eier oder der Frühstücksspeck gebraten werden. Dieser Code startet jedoch keine anderen Aufgaben. Sie würden weiterhin das Brot in den Toaster stecken und das Gerät so lange anstarren, bis das Brot ausgeworfen wird. Aber Sie wären zumindest für andere Personen ansprechbar, die Ihre Aufmerksamkeit wünschen. In einem Restaurant, in dem mehrere Bestellungen aufgegeben werden, könnte der Koch mit der Zubereitung eines weiteren Frühstücks beginnen, während das erste Frühstück zubereitet wird.

Jetzt wird der Thread für die Frühstückszubereitung nicht blockiert, während er auf gestartete Aufgaben wartet, die noch nicht abgeschlossen sind. Bei einigen Anwendungen reicht diese Änderung bereits aus. Alleine diese Änderung führt bereits dazu, dass eine GUI-Anwendung weiterhin auf den Benutzer reagiert. In diesem Szenario möchten Sie jedoch mehr. Die einzelnen Komponenten bzw. Aufgaben sollen nicht sequenziell ausgeführt werden. Es ist besser, die einzelnen Komponenten/Aufgaben zu starten, ohne auf den Abschluss der vorherigen Aufgabe zu warten.

Aufgaben gleichzeitig starten

In vielen Szenarios möchten Sie mehrere voneinander unabhängige Aufgaben unverzüglich starten. Sobald eine der Aufgabe abgeschlossen ist, können Sie dann andere Aufgaben fortsetzen, die bereit sind. Um beim Beispiel des Frühstücks zu bleiben, würden Sie dieses sehr viel schneller zubereiten können. Außerdem können Sie alle Aufgaben nahezu gleichzeitig fertigstellen. Und erhalten so ein warmes Frühstück.

System.Threading.Tasks.Task und verwandte Typen sind Klassen, mit denen Sie Aufgaben analysieren können, die gerade ausgeführt werden. So können Sie Code schreiben, der sehr viel stärker der Art und Weise ähnelt, wie Sie wirklich ein Frühstück zubereiten. Sie würden gleichzeitig mit der Zubereitung von Eiern, Frühstücksspeck und Toast beginnen. Da hierfür jeweils eine Aktion erfor-

derlich ist, würden Sie Ihre Aufmerksamkeit auf diese Aufgabe lenken, sich um die nächste Aktion kümmern und dann auf etwas anderes warten, das Ihre Aufmerksamkeit erfordert.

Sie starten eine Aufgabe und behalten dann das Task-Objekt bei, das die Arbeit repräsentiert. Sie warten jede Aufgabe ab („await“), bevor Sie mit ihrem Ergebnis arbeiten.

Lassen Sie uns nun die entsprechenden Änderungen an dem Code für das Frühstück vornehmen. Der erste Schritt besteht darin, die Aufgaben für Vorgänge bei deren Start zu speichern, anstatt auf sie zu warten:

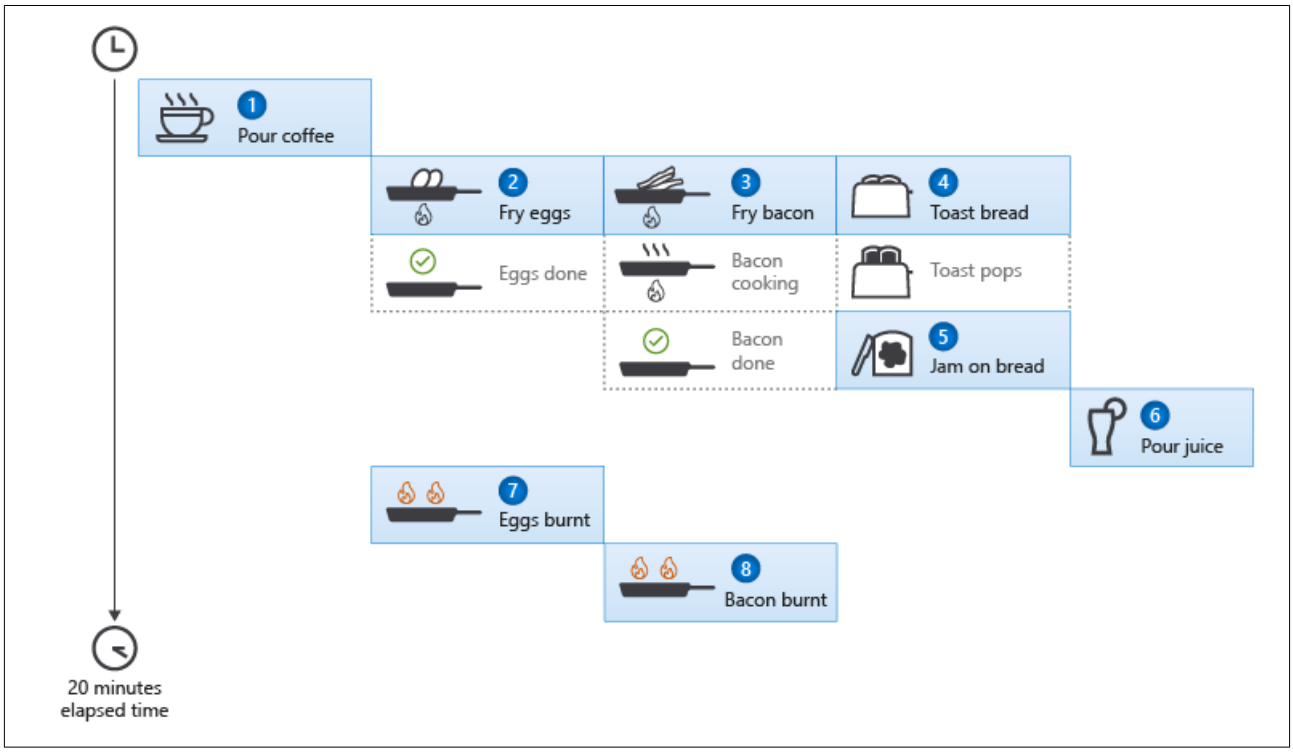
```
static async Task Main(string[] args) {

    Stopwatch sw = new Stopwatch();
    sw.Start();

    Coffee cup = PourCoffee();
    Console.WriteLine("coffee is ready");

    Task<Egg> eggsTask = FryEggsAsync(2);
    Egg eggs = await eggsTask;
    Console.WriteLine("eggs are ready");

    Task<Bacon> baconTask = FryBaconAsync(3);
    Bacon bacon = await baconTask;
    Console.WriteLine("bacon is ready");

    Task<Toast> toastTask = ToastBreadAsync(2);
    Toast toast = await toastTask;
    ApplyButter(toast);
    ApplyJam(toast);
    Console.WriteLine("toast is ready");

    Juice oj = PourOJ();
    Console.WriteLine("oj is ready");

    sw.Stop();

    Console.WriteLine($"Breakfast is ready: {sw.ElapsedMilliseconds} ms");
}
```

Starten Sie das Programm erneut. Wir haben noch immer keine Verbesserung erzielt.

Als Nächstes können Sie die await-Anweisungen für den Frühstücksspeck und die Eier an das Ende der Methode, vor dem Servieren des Frühstücks, verschieben:

```
static async Task Main(string[] args) {

    Stopwatch sw = new Stopwatch();
    sw.Start();

    Coffee cup = PourCoffee();
    Console.WriteLine("coffee is ready");

    Task<Egg> eggsTask = FryEggsAsync(2);
    Task<Bacon> baconTask = FryBaconAsync(3);
    Task<Toast> toastTask = ToastBreadAsync(2);

    Toast toast = await toastTask;
    ApplyButter(toast);
    ApplyJam(toast);
    Console.WriteLine("toast is ready");
    Juice oj = PourOJ();
    Console.WriteLine("oj is ready");

    Egg eggs = await eggsTask;
    Console.WriteLine("eggs are ready");
    Bacon bacon = await baconTask;
    Console.WriteLine("bacon is ready");
```

```
    sw.Stop();

    Console.WriteLine($"Breakfast is ready: {sw.ElapsedMilliseconds} ms");
}
```



Wir haben endlich eine Verbesserung bei der Gesamtzeit erzielt. Es hat ungefähr 20 Minuten gedauert, das Frühstück asynchron zuzubereiten, da einige Aufgaben gleichzeitig ausgeführt werden konnten.

Der obige Code funktioniert besser. Sie starten alle asynchronen Aufgaben gleichzeitig. Sie verwenden „await“ nur für Aufgaben, wenn Sie deren Ergebnisse benötigen. Der obige Code ähnelt beispielsweise Code in einer Webanwendung, die verschiedene Microservices anfordert und dann die Ergebnisse auf einer einzigen Seite zusammenfasst. Sie führen alle Anforderungen sofort aus, warten dann aber mit await auf alle diese Aufgaben und stellen die Webseite zusammen.

Kombination mit Aufgaben

Sie haben alles, was zum Frühstück benötigt wird, gleichzeitig fertig, mit Ausnahme des Toasts. Die Zubereitung des Toasts ist eine Kombination aus einem asynchronen Vorgang (das Toasten des Brotes) und synchronen Vorgängen (das Bestreichen mit Butter und Marmelade). Die Aktualisierung dieses Codes veranschaulicht ein wichtiges Konzept:

Die Kombination aus einem asynchronen Vorgang gefolgt von einer synchronen Tätigkeit ergibt einen asynchronen Vorgang. Mit anderen Worten: Wenn ein Teil eines Vorgangs asynchron ist, ist der gesamte Vorgang asynchron.

Der obige Code zeigt, dass Sie Task- oder Task<TResult>-Objekte verwenden können, um laufende Aufgaben beizubehalten. Sie warten mit „await“ auf jede Aufgabe, bevor Sie deren Ergebnis verwenden. Der nächste Schritt besteht im Erstellen von Methoden, die die Kombination anderer Tätigkeiten darstellen. Bevor Sie das Frühstück servieren, möchten Sie auf die Aufgabe warten, die für das Toasten des Brotes steht, bevor Sie das getoastete Brot mit Butter und Marmelade bestreichen. Dies können Sie mit dem folgenden Code darstellen:

```
static async Task<Toast> MakeToastWithButterAndJamAsync(int number)
{
    var toast = await ToastBreadAsync(number);
    ApplyButter(toast);
    ApplyJam(toast);

    return toast;
}
```

Die obige Methode verfügt in ihrer Signatur über den Modifizierer async. Dies signalisiert dem Compiler, dass diese Methode eine await-Anweisung enthält. Sie enthält also asynchrone Vorgänge. Diese Methode steht für die Aufgabe, bei der das Brot getoastet und dann mit Butter und Marmelade bestrichen wird. Diese Methode gibt das Ergebnis Task<TResult> aus, d. h. die Kombination dieser drei Vorgänge. Der Hauptcodeblock sieht jetzt wie folgt aus:

```
static async Task Main(string[] args) {

    Stopwatch sw = new Stopwatch();
    sw.Start();

    Coffee cup = PourCoffee();
    Console.WriteLine("coffee is ready");

    var eggsTask = FryEggsAsync(2);
    var baconTask = FryBaconAsync(3);
    var toastTask = MakeToastWithButterAndJamAsync(2);

    var eggs = await eggsTask;
    Console.WriteLine("eggs are ready");

    var bacon = await baconTask;
    Console.WriteLine("bacon is ready");

    var toast = await toastTask;
    Console.WriteLine("toast is ready");

    Juice oj = PourOJ();
    Console.WriteLine("oj is ready");

    sw.Stop();

    Console.WriteLine($"Breakfast is ready: {sw.ElapsedMilliseconds} ms");
}
```

Die obige Änderung veranschaulicht eine wichtige Technik für das Arbeiten mit asynchronem Code. Sie erstellen Aufgaben, indem Sie die Vorgänge in eine neue Methode unterteilen, die eine Aufgabe zurückgibt. Sie können entscheiden, wann auf diese Aufgabe gewartet werden soll. Sie können andere Aufgaben gleichzeitig starten.

Effizient auf Aufgaben warten

Die Reihe der await-Anweisungen am Ende des obigen Codes kann mithilfe der Methoden der Task-Klasse verbessert werden. Eine dieser APIs ist WhenAll, sie gibt eine Task zurück, die abgeschlossen wird, wenn alle Aufgaben in ihrer Argumentenliste abgeschlossen sind. Dies zeigt der folgende Code:

```
await Task.WhenAll(eggsTask, baconTask, toastTask);
Console.WriteLine("eggs are ready");
Console.WriteLine("bacon is ready");
Console.WriteLine("toast is ready");
```

Eine weitere Möglichkeit ist die Verwendung von WhenAny. Dies gibt eine Task<Task> zurück, die abgeschlossen wird, wenn eines ihrer Argumente abgeschlossen wird. Sie können mit „await“ auf die zurückgegebene Aufgabe warten – in dem Wissen, dass sie bereits abgeschlossen ist. Der folgende Code zeigt, wie Sie WhenAny verwenden können, um mit „await“ auf den Abschluss der

ersten Aufgabe zu warten und dann deren Ergebnis zu verarbeiten. Nach dem Verarbeiten des Ergebnisses der abgeschlossenen Aufgabe können Sie diese abgeschlossene Aufgabe aus der Liste der an WhenAny übergebenen Aufgaben entfernen.

```
var breakfastTasks = new List<Task> { eggsTask, baconTask, toastTask };
while (breakfastTasks.Count > 0)
{
    Task finishedTask = await Task.WhenAny(breakfastTasks);
    if (finishedTask == eggsTask)
    {
        Console.WriteLine("eggs are ready");
    }
    else if (finishedTask == baconTask)
    {
        Console.WriteLine("bacon is ready");
    }
    else if (finishedTask == toastTask)
    {
        Console.WriteLine("toast is ready");
    }
    breakfastTasks.Remove(finishedTask);
}
```

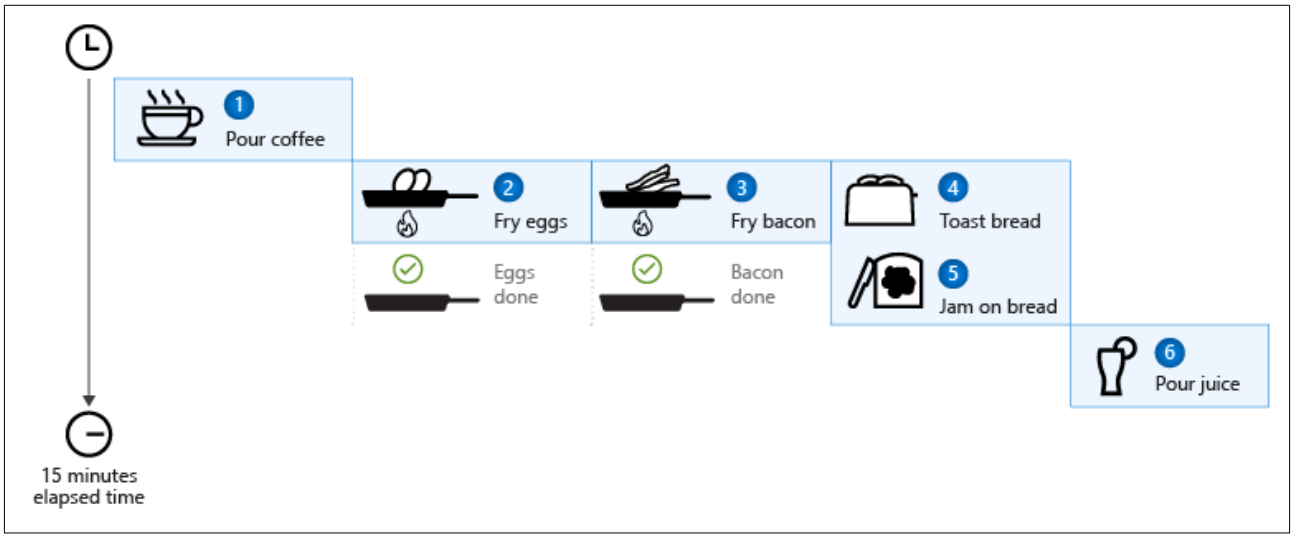
Für diesen Code benötigen Sie noch folgende using-Anweisung:

```
using System.Collections.Generic;
```

Nach allen diesen Änderungen sieht der endgültige Code folgendermaßen aus:

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Threading.Tasks;

namespace AsyncBreakfast {
    class Program {
        static async Task Main(string[] args) {

            Stopwatch sw = new Stopwatch();
            sw.Start();

            Coffee cup = PourCoffee();
            Console.WriteLine("coffee is ready");

            var eggsTask = FryEggsAsync(2);
            var baconTask = FryBaconAsync(3);
            var toastTask = MakeToastWithButterAndJamAsync(2);

            var breakfastTasks = new List<Task> { eggsTask, baconTask, toastTask };
            while (breakfastTasks.Count > 0) {
                Task finishedTask = await Task.WhenAny(breakfastTasks);
                if (finishedTask == eggsTask) {
                    Console.WriteLine("eggs are ready");
                }
                else if (finishedTask == baconTask) {
                    Console.WriteLine("bacon is ready");
                }
                else if (finishedTask == toastTask) {
                    Console.WriteLine("toast is ready");
                }
                breakfastTasks.Remove(finishedTask);
            }

            Juice oj = PourOJ();
            Console.WriteLine("oj is ready");

            sw.Stop();

            Console.WriteLine($"Breakfast is ready: {sw.ElapsedMilliseconds} ms");
        }
```

```
        static async Task<Toast> MakeToastWithButterAndJamAsync(int number) {
            var toast = await ToastBreadAsync(number);
            ApplyButter(toast);
            ApplyJam(toast);
            return toast;
        }

        private static Juice PourOJ() {
            Console.WriteLine("Pouring orange juice");
            return new Juice();
        }

        private static void ApplyJam(Toast toast) {
            Console.WriteLine("Putting jam on the toast");
        }

        private static void ApplyButter(Toast toast) {
            Console.WriteLine("Putting butter on the toast");
        }

        private static async Task<Toast> ToastBreadAsync(int slices) {
            for (int slice = 0; slice < slices; slice++) {
                Console.WriteLine("Putting a slice of bread in the toaster");
            }
            Console.WriteLine("Start toasting...");
            await Task.Delay(3000);
            Console.WriteLine("Remove toast from toaster");
            return new Toast();
        }

        private static async Task<Bacon> FryBaconAsync(int slices) {
            Console.WriteLine($"putting {slices} slices of bacon in the pan");
            Console.WriteLine("cooking first side of bacon...");
            await Task.Delay(3000);
            for (int slice = 0; slice < slices; slice++) {
                Console.WriteLine("flipping a slice of bacon");
            }
            Console.WriteLine("cooking the second side of bacon...");
            await Task.Delay(3000);
            Console.WriteLine("Put bacon on plate");
            return new Bacon();
        }

        private static async Task<Egg> FryEggsAsync(int howMany) {
            Console.WriteLine("Warming the egg pan...");
            await Task.Delay(3000);
            Console.WriteLine($"cracking {howMany} eggs");
            Console.WriteLine("cooking the eggs ...");
            await Task.Delay(3000);
            Console.WriteLine("Put eggs on plate");
            return new Egg();
        }

        private static Coffee PourCoffee() {
            Console.WriteLine("Pouring coffee");
            return new Coffee();
        }
    }
}
```



Die finale Version des asynchron zubereiteten Frühstücks hat etwa 15 Minuten in Anspruch genommen. Das liegt daran, dass einige Aufgaben gleichzeitig ausgeführt werden konnten und der Code mehrere Aufgaben gleichzeitig überwachen und nur bei Bedarf eingreifen konnte.

Dieser letzte Code ist asynchron. Er spiegelt genauer wider, wie ein Mensch ein Frühstück zubereiten würde. Vergleichen Sie den obigen Code mit dem ersten Codebeispiel in diesem Artikel. Die Kernaktionen sind beim Lesen des Codes noch immer deutlich erkennbar. Sie können diesen Code in derselben Weise lesen wie die Anweisungen für die Zubereitung eines Frühstücks am Anfang dieses Artikels. Die Sprachfunktionen für async und await stellen die Übersetzung bereit, die jede Person vornimmt, um diese schriftlichen Anweisungen zu befolgen: Starten Sie Aufgaben, sobald Sie dies können, und blockieren Sie nicht den weiteren Fortgang, indem Sie auf den Abschluss von Aufgaben warten.